

Implement a Pre-Trained CNN model as a Feature Extractor using Transfer Learning Model

Aim : To implement a Pre-Trained CNN model as a Feature Extractor using Transfer Learning Model

Objective :

- * To understand how a CNN pre-trained on ImageNet can extract visual features
- * To freeze convolutional layers and use their output features for new tasks.
- * To train only the classifier head for faster and more accurate learning

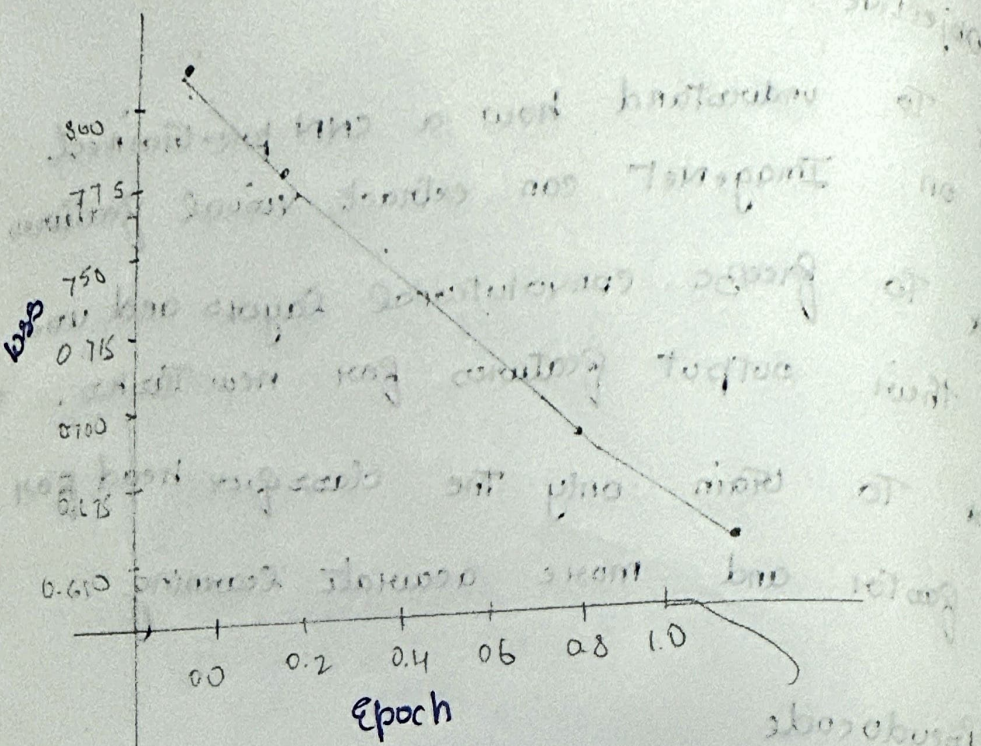
Pseudocode

- 1) Import libraries
- 2) Load a pre-trained CNN
- 3) Freeze all convolution layers to prevent retrained
- 4) Replace final classifier layer for new number of classes
- 5) Load small dataset (CIFAR-10)
- 6) Extract features
- 7) Train only new classifier head
- 8) Evaluate accuracy.

Output

Epoch [1/2], Loss: 0.8107

Epoch [2/2], Loss: 0.6423



Accuracy: 78.79%.

Observation

- * The pre-trained ResNet18 model was successfully loaded
- * Frozen layers acted as fixed features extractor, capturing low-level edges, corners, textures
- * Training was significantly faster because only the final classification parameters were updated leading to stable convergence
- * This experiment demonstrated how transfer learning can drastically reduce training time and computational cost

Result

Successfully Implemented pre-trained CNN model.



```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import os
```

```
# -----
# Step 1: Define Transformations
# -----
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```





[1]



```
for epoch in range(num_epochs):  
    running_loss = 0.0  
    for images, labels in train_loader:  
        images, labels = images.to(device), labels.to(device)  
  
        optimizer.zero_grad()  
        outputs = model(images)  
        loss = criterion(outputs, labels)  
        loss.backward()  
        optimizer.step()  
        running_loss += loss.item()  
  
    avg_loss = running_loss / len(train_loader)  
    train_loss_list.append(avg_loss)  
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {avg_loss:.4f}")  
  
print("\n🟢 Training completed successfully!")  
  
# =====
```





[1]



```
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 10) # CIFAR-10 has 10 classes

# Move model to device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# -----
# Step 4: Define Loss and Optimizer
# -----
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.fc.parameters(), lr=0.001)

# -----
# Step 5: Train Only the Classifier
# -----
num_epochs = 2 # keep small for quick run
train_loss_list = []
```





[1]



```
# Download a small example dataset (CIFAR-10)
train_data = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_data = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

# -----
# Step 2: Load Pre-trained Model
# -----
model = models.resnet18(pretrained=True)

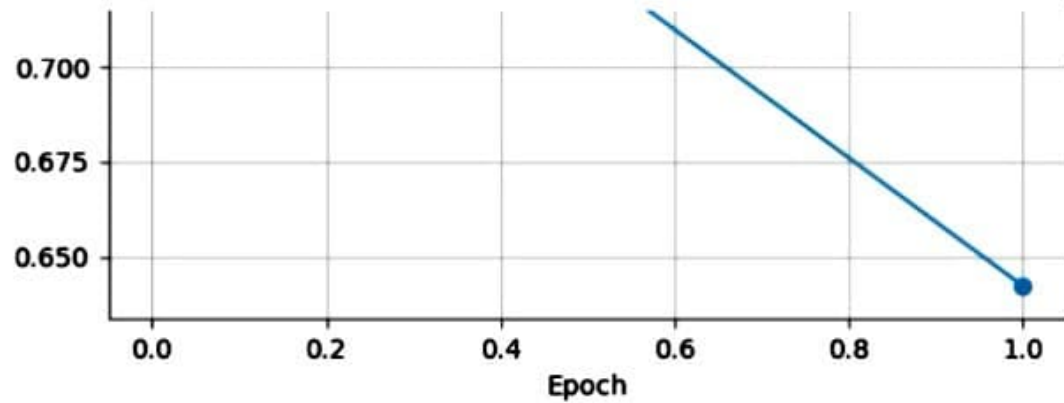
# Freeze all convolutional layers
for param in model.parameters():
    param.requires_grad = False

# -----
# Step 3: Replace the Classifier
```





[1]



Test Accuracy: 79.21%



[1]

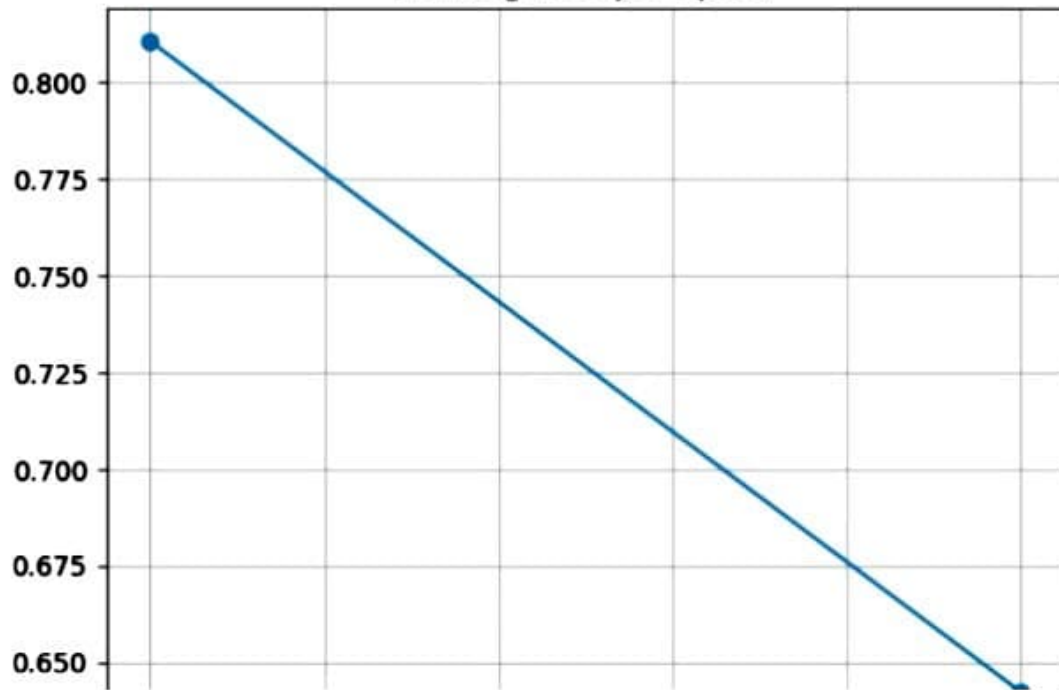
Start coding or generate with AI



[4]

Loss

Training Loss per Epoch





[1]



```
_, predicted = torch.max(outputs, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()
```

```
accuracy = 100 * correct / total
print(f"\n Test Accuracy: {accuracy:.2f}%")
```



```
100%|██████████| 170M/170M [00:10<00:00, 16.0MB/s]
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or
  warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/resnet18-f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-f3707
100%|██████████| 44.7M/44.7M [00:00<00:00, 156MB/s]
Epoch [1/2], Loss: 0.8107
Epoch [2/2], Loss: 0.6423
```

✅ Training completed successfully!





[1]



```
plt.plot(train_loss_list, marker='o')
plt.title("Training Loss per Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

# -----
# Step 7: Test Accuracy
# -----
correct = 0
total = 0
model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        total += labels.size(0)
```

